

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

Тема: Генетические алгоритмы «Решение sudoku»

Студент гр. 4343

Студент гр. 4343

Студент гр. 4343

Руководитель

А.С. Васильев

С. Овеси

В.М. Чулков

Т.Р. Жангиров

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студенты: В.М. Чулков, А.С. Васильев, С. Овеси

Группа: 4343

Тема практики: Генетические алгоритмы «Решение sudoku»

Задание на практику:

Для заданного игрового поля sudoku, где уже заданы некоторые цифры (размеры 9x9), необходимо разместить цифры от 1 до 9, так, чтобы они не нарушали правил.

Сроки прохождения практики: 06.07.2026 – 17.07.2026

Дата сдачи отчета: 16.07.2026

Дата защиты отчета: 17.07.2026

Студент гр. 4343	_____	А.С. Васильев
Студент гр. 4343	_____	С. Овеси
Студент гр. 4343	_____	В.М. Чулков
Руководитель	_____	Т.Р. Жангиров

АННОТАЦИЯ

В данной работе рассматривается процесс разработки программного приложения на языке Python для решения головоломки судоку с применением генетических алгоритмов. Реализован оптимизированный вычислительный алгоритм, включающая скрещивание по блокам 3x3, мутацию, динамическое изменение шанса мутации и механизм обмена особями между популяциями для эффективного выхода из локальных экстремумов. Для обеспечения интерактивного взаимодействия с алгоритмом спроектирован графический пользовательский интерфейс на базе библиотек streamlit и plotly. Разработанный интерфейс поддерживает различные способы ввода начальных данных (случайная генерация с выбором сложности, ручной ввод, чтение из файла), позволяет настраивать параметры генетического алгоритма и визуализирует процесс поиска решения в реальном времени посредством вывода метрик и графиков сходимости популяций.

SUMMARY

This paper examines the development of a Python software application for solving Sudoku puzzles using genetic algorithms. An optimized computational algorithm is implemented, including 3x3 crossover, mutation, dynamic mutation chance, and a mechanism for exchanging individuals between populations to effectively overcome local extremes. To facilitate interactive interaction with the algorithm, a graphical user interface based on the streamlit and plotly libraries is designed. The developed interface supports various input methods (random generation with difficulty selection, manual input, reading from a file), allows for configuring genetic algorithm parameters, and visualizes the solution search process in real time by displaying metrics and population convergence graphs.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	5
1 ПОСТАНОВКА ЗАДАЧИ.....	6
1.1 Предметная область.....	6
1.2 Задачи практики.....	6
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	7
2.1. Реализация генетического алгоритма.....	7
2.2. Создание графического интерфейса.....	8
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ.....	12
4 ОТЗЫВ РУКОВОДИТЕЛЯ.....	13
ЗАКЛЮЧЕНИЕ.....	14
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	15

ВВЕДЕНИЕ

Основной предметной областью практики являются методы применения генетических алгоритмов для решения задач, а также разработка графических пользовательских интерфейсов для наглядной визуализации вычислительных процессов.

Цель практики - создание приложения с графическим интерфейсом для настройки, запуска и визуализации работы генетического алгоритма, решающего задачу sudoku в реальном времени.

Для достижения цели необходимо решить следующие задачи: изучить принципы работы генетических алгоритмов и построения GUI; спроектировать интерфейс с элементами управления и визуализацией игрового поля; реализовать логику генетического алгоритма; связать графический интерфейс с алгоритмом; провести тестирование и отладку готового приложения.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Генетические алгоритмы — эвристические алгоритмы поиска, используемые для решения задач оптимизации. Основой этих алгоритмов являются операции мутации, скрещивания и случайного подбора значений.

GUI — графический интерфейс пользователя, предоставляющий информацию о работе программы.

1.2 Задачи практики

1. Необходимо реализовать генетический алгоритм поиска решения задачи Судоку. — Ответственный: В.М. Чулков
2. Работа алгоритма должна быть представлена в специально разработанном GUI, с помощью которого пользователь может задавать необходимые параметры запуска и свободно отслеживать каждый этап работы алгоритма. — Ответственный: А.С. Васильев
3. Реализовать форматированный ввод и вывод матрицы судоку для удобной обработки графическим интерфейсом. Для заданных матриц разработать алгоритм заполнения пустых клеток случайными числами для работы генетического алгоритма. — Ответственный: О. Саялел

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Реализация генетического алгоритма

Проведен анализ задачи с сопутствующим изучением теоретического материала по написанию генетических алгоритмов.

В качестве коэффициента приспособленности отдельной особи выбрано кол-во цифр в матрице, которые не удовлетворяют правилам sudoku. Соответственно, коэффициент приспособленности показывает кол-во ошибок в матрице данной особи, и чем он меньше тем ближе особь к локальному экстремуму.

В качестве предварительной оптимизации следует не обрабатывать заданные изначально клетки, а также на этапе инициализации матрицы sudoku заполнить все клетки с учетом фиксированных значений, то есть исключить ошибки в матрице между фиксированными значениями и добавленными случайно при инициализации.

Задача sudoku при решении генетическим алгоритмом может очень часто приводить к локальным экстремумам, ошибка в которых будет маленькой, но при этом само решение будет далеко от реального, ввиду этого было принято решение использовать несколько разных популяций и заменять особь с максимальным числом ошибок особью с минимальным числом ошибок из предыдущей популяции каждые 20 поколений.

Операция мутации как замена имеющегося значения на случайное в рамках этой задачи будет часто приводить к ошибкам (если для данной клетки ошибок не было, то операция на ней обязательно их создаст), поэтому в качестве мутации можно использовать перестановку имеющегося числа в клетке с числом в другой клетке. Если такая мутация происходит между клетками из квадрата 3×3 , то соответственно в рамках пространства отношения данных клеток ошибки возникать не будет (но соответственно в двух других пространствах она также может возникнуть).

Для оптимизации алгоритма, для мутации выбираются только те блоки 3x3 в которых заведомо известно, что есть ошибка.

Операции скрещивания (также как и мутации) стоит проводить по квадратам 3x3. Это будет предотвращать возможность появления дополнительных ошибок.

На третьем этапе работы были внесены дополнительные оптимизации алгоритма с учетом комментариев руководителя.

Если минимальное значение Fitness (число ошибок особи) в данной популяции в данной популяции не меняется на протяжении 10 поколений, то шанс мутации удваивается относительно начального.

Если минимальное значение Fitness (кол-во ошибок особи) в данной популяции не меняется на протяжении 50 поколений, то популяция удаляет всех особей, за исключением двух особей с минимальным значением Fitness, после чего формирует новых кандидатов.

Исправлена ошибка генерации случайных значений в матрице, создающая неправильные 3x3 сегменты (в подматрицах 3x3 могло нарушаться правило, что недопустимо с выбранными способами скрещивания и мутации)

Модернизирована функция мутации, вместо случайного сегмента 3x3 выбирается случайный сегмент заведомо содержащий ошибку и та клетка которая содержит ошибку обязательно мутируется

2.2. Создание графического интерфейса

Реализован шаблон графического интерфейса с возможностью задания кастомных параметров работы алгоритма и отслеживанием каждого шага работы. Шаблон представлен на рис. 1.

Графический интерфейс отображает исходное поле sudoku, решение с минимальным числом ошибок среди всех популяций, номер текущего поколения, минимальное и среднее значения коэффициента приспособленности..

График приспособленности показывает среднее значение приспособленности для данного поколения, среднее значение приспособленности для каждой популяции и значение приспособленности для имеющегося поколения с минимальной приспособленностью. Шаблон графика представлен на рис. 2.

В графическом интерфейсе также присутствует панель управления позволяющая перемещаться по шагам работы алгоритма, запускать алгоритм, генерировать новую задачу и получать конечное решение для сгенерированной задачи.

Ввод осуществляется тремя способами, случайная генерация, ввод из файла и ручной ввод. В данный момент полноценно реализовано случайная генерация матрицы sudoku. Ручной ввод и ввод через файл представляют из себя отдельные формы.

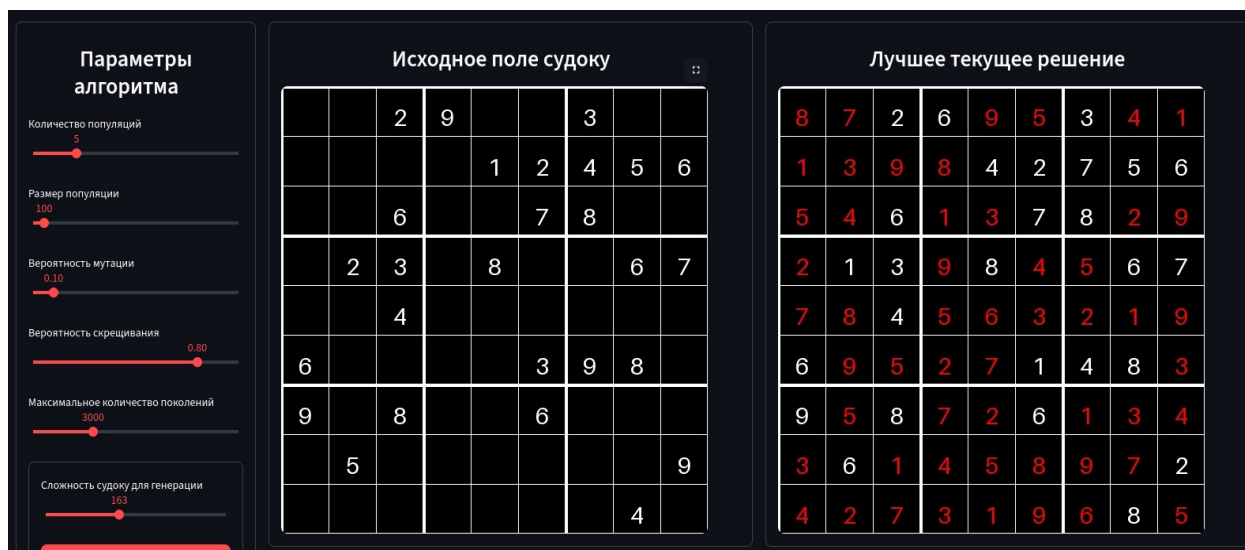


Рисунок 1 – Шаблон графического интерфейса

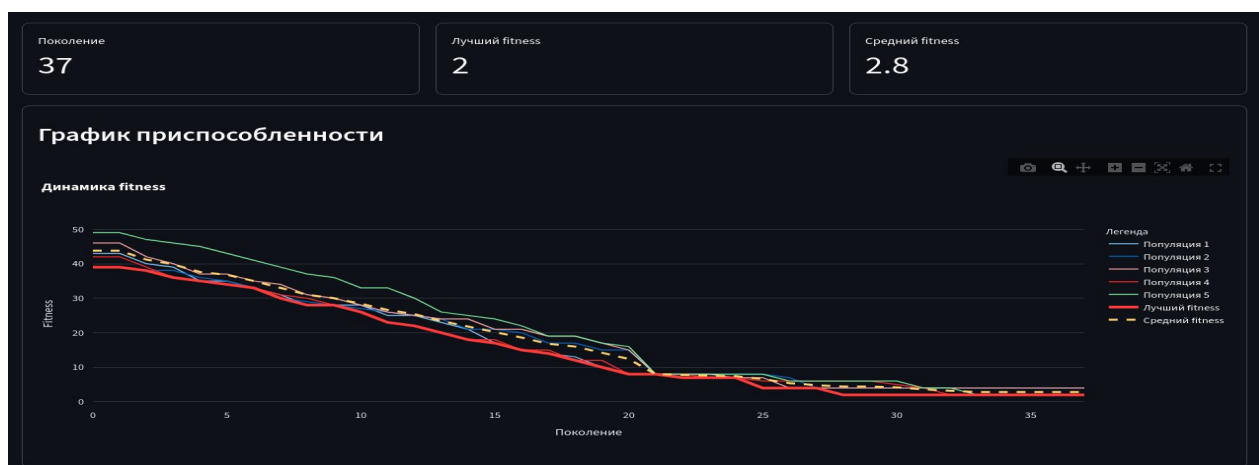


Рисунок 2 – Шаблон графика приспособленности для каждого поколения

На втором этапе работы графический интерфейс был обновлен, появилась возможность перехода между поколениями текущего алгоритма, обновлена отрисовка цифр внутри блоков, фиксированные по условию отображаются белыми, решение отображается красными символами. Привязаны кнопки GUI, доведены до рабочего состояния.

Шаблон отрисовки графика заменен на рабочую функцию, которая строит график по точкам через каждые 50 поколений. Текущий графический интерфейс изображён на рис. 3.

На третьем этапе работы был реализован способ задания матрицы sudoku вручную и чтение задачи из файла. Также изменен график отображения работы алгоритма с учетом комментариев руководителя (цвета стали более контрастными и для каждой популяции создается отдельный график). Также появилась возможность выбора сложности генерируемой задачи. Сложность варьируется от 0 до 400, где 0 это самая простая задача, а 400 — самая сложная. Также ползунки вероятностей теперь могут задавать значения полноценно от 0 до 1, а не в ограниченных промежутках данного отрезка. Текущий графический интерфейс изображён на рис. 4.



Рисунок 3 — Графический интерфейс программы на 2 шаге

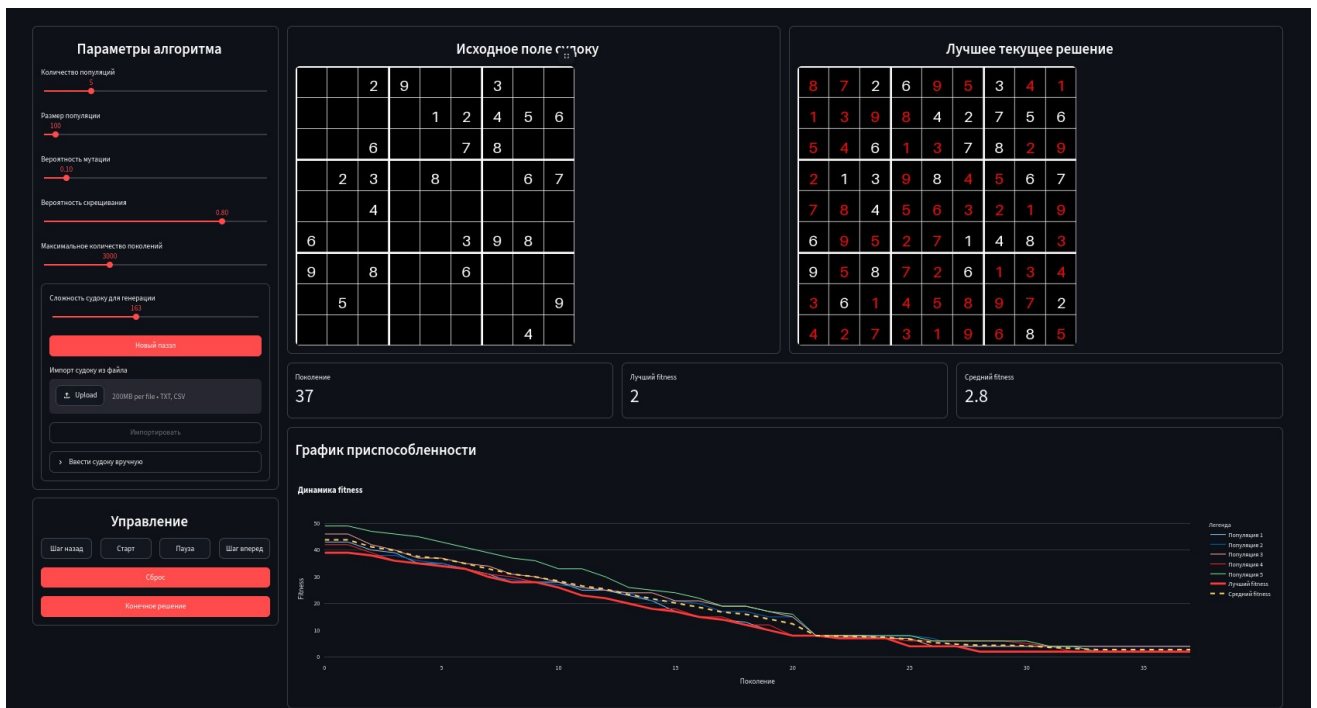


Рисунок 4 — Графический интерфейс программы на 3 шаге

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

Для выполнения задач учебной практики были использованы разные технологии предоставляемые языком программирования Python [1-2].

Графический интерфейс реализован при помощи библиотек streamlit и plotly [3].

Генерация случайной матрицы осуществляется при помощи библиотеки dokusan [4]. Перевод матрицы sudoku из генератора объекта библиотеки dokusan в удобный для обработки вид осуществляется через библиотеку numpy.

Перевод матрицы в формат изображения осуществляется при помощи библиотеки pillow [5].

4 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на Отлично.

ЗАКЛЮЧЕНИЕ

В ходе выполнения практической работы было реализовано полноценное программное решение задачи решения головоломки судoku.

Решение задачи находится при помощи генетического алгоритма объединяющего в своей логике базовые и модифицированные оптимизационные подходы, основными из которых являются: турнирные методы поиска родителей, мутация сегментов с худшим показателем приспособленности, динамическая вероятность мутации и обмен особей между популяциями.

Реализован графический интерфейс обеспечивающий полнофункциональную настройку параметров работы алгоритма для решения случайно сгенерированной или заданной пользователем задачи. Для отслеживания работы алгоритма реализован вывод метрики и графическое представление изменения состояния всех популяций для каждого поколения в общем и частном виде.

В ходе работы было полноценно реализовано заполнение заданных матриц судoku с учетом тонкостей работы генетического алгоритма для оптимизации скорости и точности его выполнения. Также был написан удобный формат передачи данных обеспечивающий сообщение алгоритма и графического интерфейса.

Разработанная система успешно прошла тестирование на головоломках различной сложности.

Ссылка на сходный код программы: <https://github.com/circonfl3x/gui-algorithm-visualizer-4343-sss>

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Эйял Вирсански., Генетические алгоритмы на Python / пер. с англ. А. А. Слинкина. – М.:ДМК Пресс, 2020. – 286 с.: ил.// Copyright © Packt Publishing, 2020.
2. Панченко, Т. В. Генетические алгоритмы : учебно-методическое пособие / под ред. Ю. Ю. Тарасевича. — Астрахань : Издательский дом «Астраханский университет», 2007. — 87 с. // © Панченко Т. В., 2007, © Издательский дом «Астраханский университет», 2007
3. Streamlit documentation [Электронный ресурс]. URL: <https://docs.streamlit.io/> (дата обращения: 10.07.2026).
4. Dokusan documentation [Электронный ресурс]. URL: <https://pypi.org/project/dokusan/> (дата обращения: 09.07.2026).
5. Pillow документация [Электронный ресурс]. — <https://pillow.readthedocs.io/en/stable> (дата обращения: 09.07.2026).